

PSET2

1 – Helper Functions

1.1 – Determinar si el jugador gana

¿Qué hace? Determina si el jugador ya adivinó todas las letras del `secret_word`.

Mi aproximación

```
1 def has_player_won(secret_word, letters_guessed):
2     """
3     secret_word: string, the lowercase word the user is guessing
4     letters_guessed: list (of lowercase letters), the letters that have
5     been
6         guessed so far
7     returns: boolean, True if all the letters of secret_word are in
8     letters_guessed,
9     False otherwise
10    """
11    for letter in secret_word:
12        if letter not in letters_guessed:
13            return False
14    return True
```

Como los `String` son iterables puedo recorrerlos, entonces verifico si cada letra en `secret_word` esta en `letters_guessed`.

1.2 – Revelando letras

¿Qué hace? Devuelve un `string` donde:

- Las letras adivinadas aparecen.
- Las no adivinadas están representadas por asteriscos (`*`).

Mi aproximación

```
1 def get_word_progress(secret_word, letters_guessed):
2     """
3     secret_word: string, the lowercase word the user is guessing
```

```

4     letters_guessed: list (of lowercase letters), the letters that have
    been
5         guessed so far
6
7     returns: string, comprised of letters and asterisks (*) that
    represents
8         which letters in secret_word have not been guessed so far
9     """
10    word = ""
11
12    for i in range(len(secret_word)):
13        if secret_word[i] not in letters_guessed:
14            word += "*"
15        else:
16            word += secret_word[i]
17    return word

```

1.3 – Obtener letras disponibles

¿Qué hace? Devuelve las letras del alfabeto que aún no fueron adivinadas, en orden alfabético.

Mi aproximación

```

1    def get_available_letters(letters_guessed):
2        """
3        letters_guessed: list (of lowercase letters), the letters that have
    been
4            guessed so far
5
6        returns: string, comprised of letters that represents which
7            letters have not yet been guessed. The letters should be returned in
8            alphabetical order
9        """
10
11    result = ""
12
13    for char in string.ascii_lowercase:
14        if char in letters_guessed:
15            continue
16        result += char
17    return result

```

2 – El juego

2.1 – Inicio del juego

- Imprimí un mensaje de bienvenida.

- Muestra cuántas letras tiene el `secret_word`.
- Inicia con 10 intentos disponibles.

2.2 – Cada turno del juego debe incluir:

- Un separador visual (al menos `-----`).
- Cuántos intentos quedan.
- Letras disponibles (usando `get_available_letters`).
- Un `input()` para que el usuario ingrese una letra.
- Feedback inmediato: ¿está en la palabra o no?
- El progreso actual (usando `get_word_progress`).

2.3 – Lógica de los intentos:

- Si el usuario adivina correctamente, no pierde intentos.
- Si adivina mal:
 - Si es **consonante**, pierde 1 intento.
 - Si es **vocal**, pierde 2 intentos.
- Si ingresa algo inválido o una letra repetida, no pierde intentos, pero recibe una advertencia.

2.4 – Funcionalidad especial: ayuda con !

- Si el usuario escribe `!`, y el parámetro `with_help` es `True`, se revela una letra del `secret_word` aún no adivinada.
- Esto **cuesta 3 intentos**.
- Si no tiene suficientes intentos, se muestra una advertencia.
- Esto requiere una pequeña función auxiliar para seleccionar aleatoriamente una letra faltante del secreto.

2.5 – Final del juego

- Si el jugador adivina toda la palabra → Felicitaciones y mostrar puntaje.
- Si se queda sin intentos → Se muestra la palabra secreta y mensaje de derrota.

2.6 – Puntaje final

El puntaje se calcula así:

```
1 score = (guesses_remaining + 4 * num_letras_unicas_en_palabra) + (3 * len(palabra))
```

Mi aproximación

```
1 def reveal_letter(secret_word, letters_guessed):
2     """Devuelve una letra aún no adivinada que esté en la palabra
3     secreta."""
4     options = [letter for letter in set(secret_word) if letter not in
5     letters_guessed]
6     if options:
7         return random.choice(options)
8     return None
```

```
1 def hangman(secret_word, with_help):
2     """
3     secret_word: string, the secret word to guess.
4     with_help: boolean, this enables help functionality if true.
5
6     Starts up an interactive game of Hangman.
7
8     * At the start of the game, let the user know how many
9     letters the secret_word contains and how many guesses they start
10    with.
11
12    * The user should start with 10 guesses.
13
14    * Before each round, you should display to the user how many guesses
15    they have left and the letters that the user has not yet guessed.
16
17    * Ask the user to supply one guess per round. Remember to make
18    sure that the user puts in a single letter (or help character '!'
19    for with_help functionality)
20
21    * If the user inputs an incorrect consonant, then the user loses ONE
22    guess,
23    while if the user inputs an incorrect vowel (a, e, i, o, u),
24    then the user loses TWO guesses.
25
26    * The user should receive feedback immediately after each guess
27    about whether their guess appears in the computer's word.
28
29    * After each guess, you should display to the user the
30    partially guessed word so far.
31
32    -----
33    with_help functionality
34    -----
35
36    * If the guess is the symbol !, you should reveal to the user one of
37    the
```

```

34         letters missing from the word at the cost of 3 guesses. If the user
does
35         not have 3 guesses remaining, print a warning message. Otherwise,
add
36         this letter to their guessed word and continue playing normally.
37
38         Follows the other limitations detailed in the problem write-up.
39         """
40         print("Welcome to Hangman!")
41         print(f"I am thinking of a word that is {len(secret_word)} letters
long.")
42
43         guesses = 10
44         letters_guessed = []
45
46         while not has_player_won(secret_word, letters_guessed) and guesses >
0:
47             print("-----")
48             print(f"You have {guesses} guesses left.")
49             print(f"Available letters:
{get_available_letters(letters_guessed)}")
50             user = input("Please guess a letter: ").lower()
51
52             # Ayuda con '!'
53             if user == "!" and with_help:
54                 if guesses >= 3:
55                     letra_revelada = reveal_letter(secret_word,
letters_guessed)
56                     if letra_revelada:
57                         print(f"Letter revealed: {letra_revelada}")
58                         letters_guessed.append(letra_revelada)
59                         guesses -= 3
60                     else:
61                         print("No letters left to reveal.")
62                 else:
63                     print(f"Oops! Not enough guesses left:
{get_word_progress(secret_word, letters_guessed)}")
64                     print(get_word_progress(secret_word, letters_guessed))
65                     continue
66
67             # Letra válida
68             elif user in string.ascii_lowercase and len(user) == 1:
69                 if user in letters_guessed:
70                     print(f"Oops! You've already guessed that letter:
{get_word_progress(secret_word, letters_guessed)}")
71                     continue
72
73                 letters_guessed.append(user)
74
75                 if user in secret_word:
76                     print(f"Good guess: {get_word_progress(secret_word,
letters_guessed)}")
77                 else:
78                     print(f"Oops! That letter is not in my word:
{get_word_progress(secret_word, letters_guessed)}")
79                     if user in "aeiou":

```

```
80             guesses -= 2
81         else:
82             guesses -= 1
83
84     else:
85         print(f"Oops! That is not a valid letter. Please input a
letter from the alphabet: {get_word_progress(secret_word,
letters_guessed)}")
86
87     print("-----")
88     if has_player_won(secret_word, letters_guessed):
89         unique_letters = len(set(secret_word))
90         score = (guesses + 4 * unique_letters) + (3 * len(secret_word))
91         print("Congratulations, you won!")
92         print(f"Your total score for this game is: {score}")
93     else:
94         print(f"Sorry, you ran out of guesses. The word was
{secret_word}")
```